САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

Дисциплина: Бэк-энд разработка

Отчет

Домашняя работа №4

Выполнил:

Рыбинская Злата

Группа

К3344

Проверил:
Добряков Д. И.

Санкт-Петербург

2025 г.

1 Постановка задачи

Разделите ваше монолитное бэкенд-приложение на микросервисы, придерживаясь концепции database-per-service. Спроектируйте микросервисную архитектуру: отразите взаимосвязь между микросервисами, опишите способы их взаимодействия друг с другом. Спроектируйте разделение вашей БД на обособленные БД. Спроектируйте и опишите эндпоинты, необходимые для обеспечения межсервисного взаимодействия в формате спецификации OpenAPI. Опишите варианты запросов и ответов, задокументируйте возможные ошибки. В результате вы должны сформировать документ, в котором будут описаны конкретные требования и шаги для достижения разделения вашего монолитного бэкенд-приложения на отдельные микросервисы.

2 Ход работы

Для разделения монолитного приложения используется паттерн Strangler fig. В текущем приложении можно выделить самостоятельные модули:

1) Users Service. Отвечает за: пользователей; аутентификацию; подписки.
2) Recipes Service. Отвечает за: рецепты; шаги; ингредиенты.
3) Interactions Service. Отвечает за: лайки; рейтинги; комментарии.

Каждый из этих модулей является независимым и может быть вынесен в отдельный сервис.

Каждый микросервис имеет свою базу данных (рис. 1):

1) Users DB: users, subscriptions.
2) Recipes DB: recipes, steps, ingredients, recipe_ingredients.
3) Interactions DB: likes, ratings, comments.

Сервисы не обращаются напрямую к чужим таблицам.

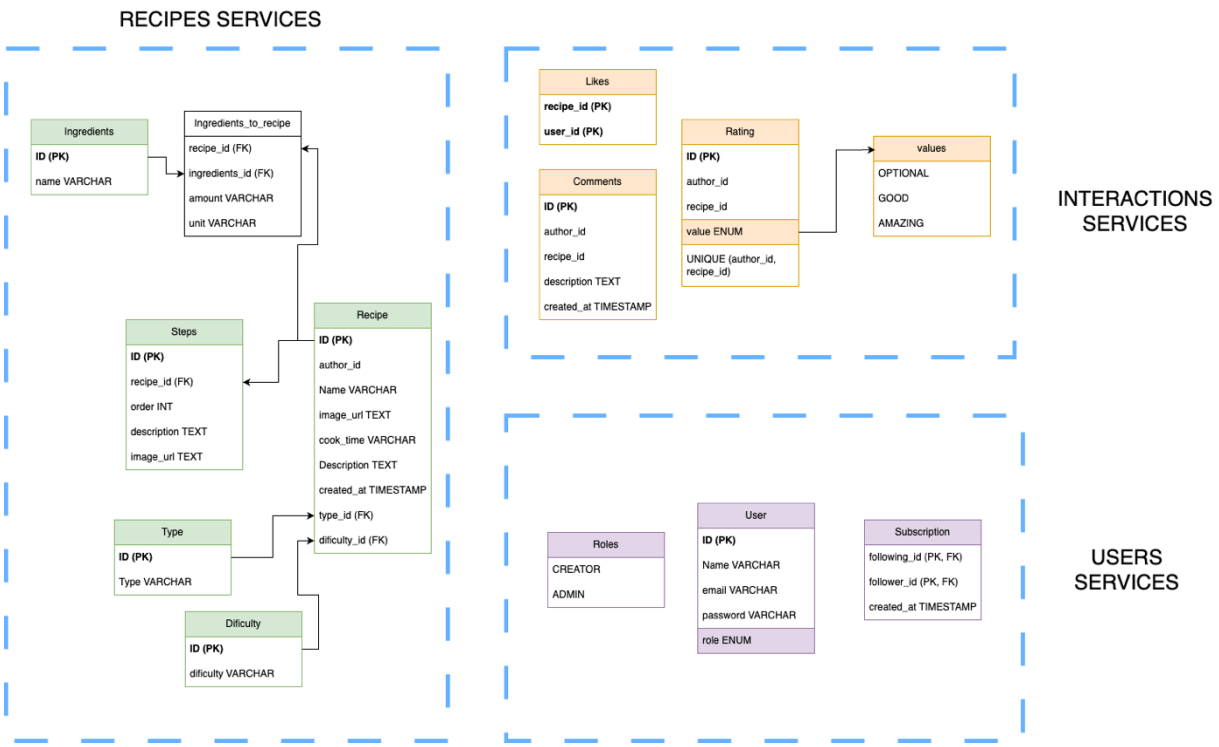

Рисунок 1 – БД, разделенная на микросервисы

Введем единые точки входа:

/api/users/...

/api/recipes/...

/api/interactions/...

Эндпоинты для межсервисного взаимодействия:

Users Service:

GET /internal/users/{id}, ответ:

{
"id": 1,
"email": "user@mail.com"
}
ошибка:

404 user not found

Recipes Service:

GET /internal/recipes/{id}, ответ:

{
"id": 1,

"name": "Пирог"
}
ошибка:

404 recipe not found

Interactions Service:

POST /internal/likes, запрос:

{
"userId": 1,
"recipeId": 2
}
ответ:

{
"id": 10,
"userId": 1,
"recipeId": 2
}

Обработка ошибок:

401 unauthorised

404 user not found

404 recipe not found

409 already liked

Шаги перехода на микросервисы:

- выделить независимые модули
- описать маршрутизацию через фасад
- настроить nginx
- создать отдельные сервисы
- вынести базы данных
- реализовать internal API
- постепенно переключать маршруты

3 Вывод

Монолитное приложение было проанализировано и разделено на независимые микросервисы по доменам. для перехода выбран паттерн Strangler fig. Он позволяет постепенно заменить монолит без остановки системы.

Каждый сервис получил свою зону ответственности и собственную базу данных. прямые связи между таблицами устранены. взаимодействие реализовано через HTTP API.

Спроектированы внутренние эндпоинты для проверки пользователей и рецептов. описаны сценарии работы и возможные ошибки.

В результате система становится гибкой. можно развивать и масштабировать сервисы отдельно. уменьшается связность и упрощается поддержка кода.